

THE HIERARCHY — who extends what

```

uvm_void
├── uvm_object      create, copy, compare
│   ├── uvm_transaction
│   │   ├── uvm_sequence_item  your transaction
│   │   │   └── uvm_sequence    your sequences
│   └── uvm_report_object
│       └── uvm_component      lives in the tree:
│                               hierarchical name,
│                               parent and phases
│   ├── uvm_test              your tests
│   ├── uvm_env               your env
│   ├── uvm_agent             your agent
│   ├── uvm_driver            your driver
│   ├── uvm_monitor           your monitors
│   ├── uvm_scoreboard        your scoreboard
│   ├── uvm_subscriber        your coverage
│   └── uvm_sequencer         as is, with a typedef

```

The line that matters: an object is created, used and thrown away; a component lives for the whole simulation. That is why one constructor is (name) and the other (name, parent).

THE SEVEN DEBUG KNOBS — six are plusargs: nothing gets recompiled

Knob	What for	The symptom it solves
VLT_TRACE=1	see the real signals	when the log is no longer enough — 47 % of the work
+UVM_VERBOSE=UVM_HIGH	see the monitors' messages	the scoreboard screams on every one : the monitor samples wrong, the DUT almost never is
+UVM_CONFIG_DB_TRACE	who put what in the config_db, and who read it	the config_db "does not find it": look at the scope of the set, not the get
+TOPOLOGY → print_topology()	the tree UVM actually built	the tree is not the one you drew: a create() without the factory, or a late override
+UVM OBJECTION_TRACE	who raised and who dropped the objection	it ends at t = 0 and says PASS, or it never ends
+UVM_TIMEOUT=5000000,N0	a ceiling for the hang, instead of waiting	the simulation hung and you do not know where
+UVM_MAX_QUIT_COUNT=N	kill the run at the Nth error	the four-gigabyte log when the scoreboard fails on every transaction

Golden rule: suspect number one is never the DUT — it is the testbench watching it.

THE NINE PHASES — only one consumes time

Phase	What for	Order
build	instantiate the components	top-down
connect	connect the ports	bottom-up
end_of_elaboration	hierarchy ready, before simulating	bottom-up
start_of_simulation	last call before time 0	bottom-up
run	task : this is where the simulation happens	one thread each
extract	gather the data from the run	bottom-up
check	decide whether it passed	bottom-up
report	print the verdict	bottom-up
final	close files and exit	top-down

build and final are the two top-down ones; build, because the parent has to exist to create the child. The rest go the other way. They are all function except run, and that is why no other one can wait for an edge.

And what decides when it ends is not run: it is the objections. raise when starting, drop when finishing, and a drain_time if something is still in flight, uncompleted.

THE HANDSHAKE — sequence ↔ driver

```

The sequence
task body();
    req = my_item::type_id
          ::create("req");
    start_item(req); // waits
    assert(req.randomize());
    finish_item(req); // waits
                      // req.result is back
endtask

```

It is a **loan**, not a copy: the sequence still holds the handle, so the driver writes the result **inside the item** before item_done(). Without that item_done() the sequence hangs in its finish_item() **without a single error message**.

```

The driver
task run_phase(uvm_phase f);
    forever begin
        seq_item_port
            .get_next_item(cmd);
        bfm.send_op(cmd.A,
                    cmd.B, cmd.op);
        seq_item_port
            .item_done();
    end
endtask

```

Coverage comes out 0 %: the covergroup is missing its new() or its sample().

All of this runs with no licences: Verilator + UVM 2020.3.1.